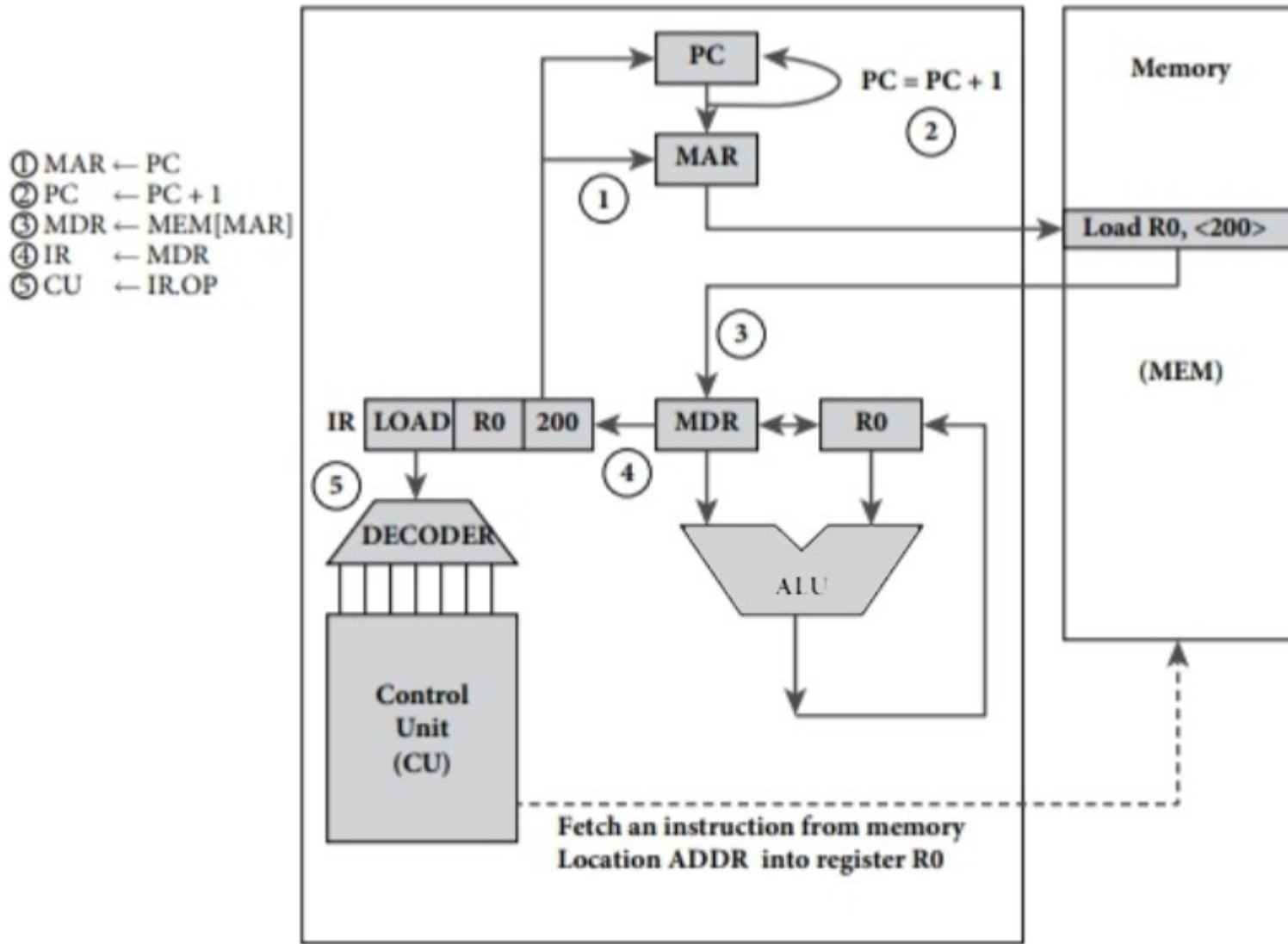




Exceptions and Interrupts

For CDA 3103C

Lecturer: Dr. John Aedo

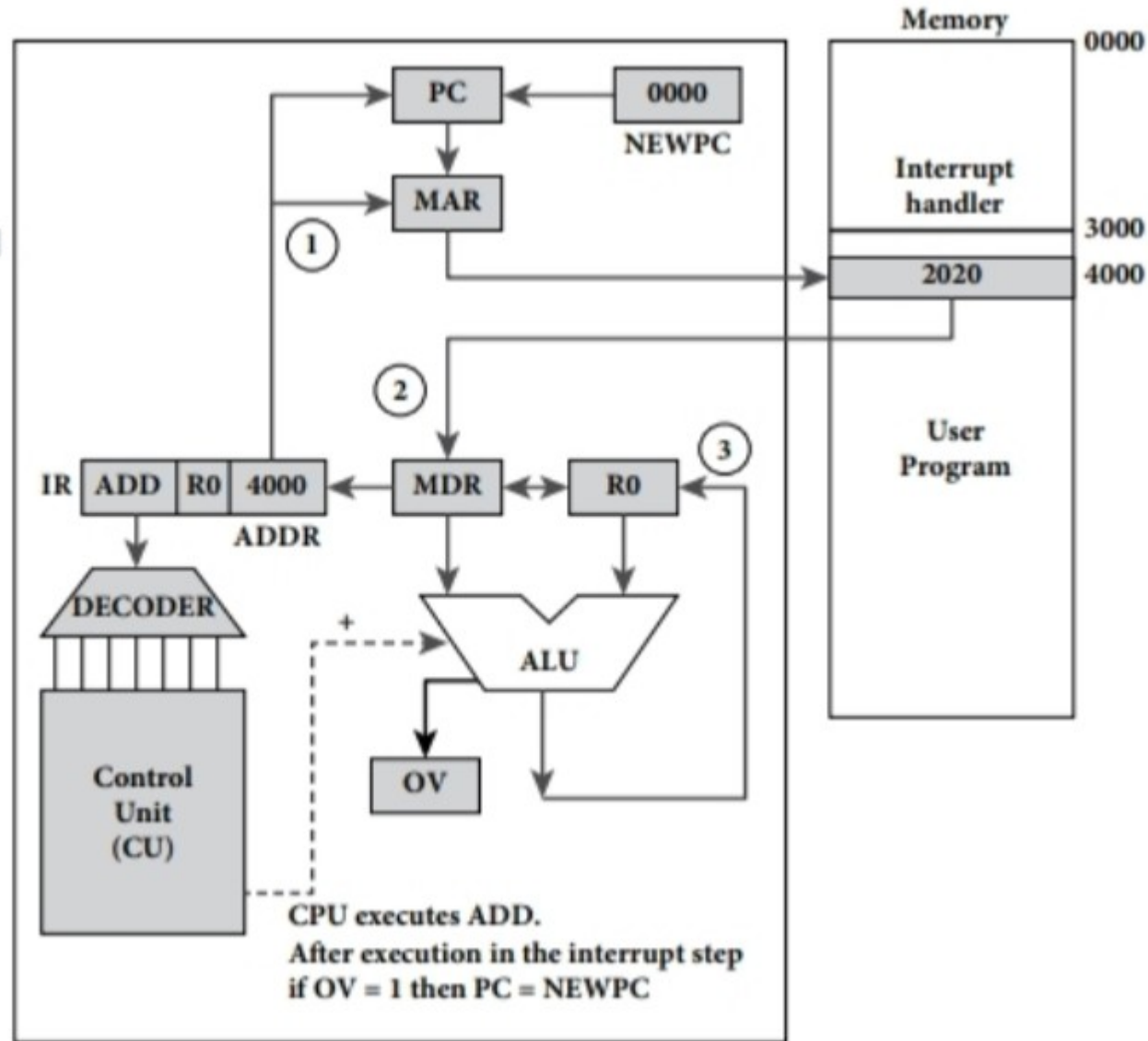


VM/0

1. Load the address of the next instruction into the MAR
2. Increment the Program Counter
3. Load the instruction into the MDR
4. Load the instruction into the Instruction register
5. Decode the Instruction
6. Execute
7. Repeat

FIGURE 8.1 Tiny VM/0 CPU executing fetch.

- ① $MAR \leftarrow OP.ADDR$
- ② $MDR \leftarrow MEM[MAR]$
- ③ $R0 \leftarrow R0 + MDR$
BACK TO FETCH



VM/0, now with Error Handling

We add two new hardware components:

1. An Overflow flip-flop that gets set when an overflow condition occurs.
2. A register (NEWPC) that stores a special memory address.

Before the next instruction, the CPU checks the OV flag. If set, it executes the instruction at NEWPC, our error handler.

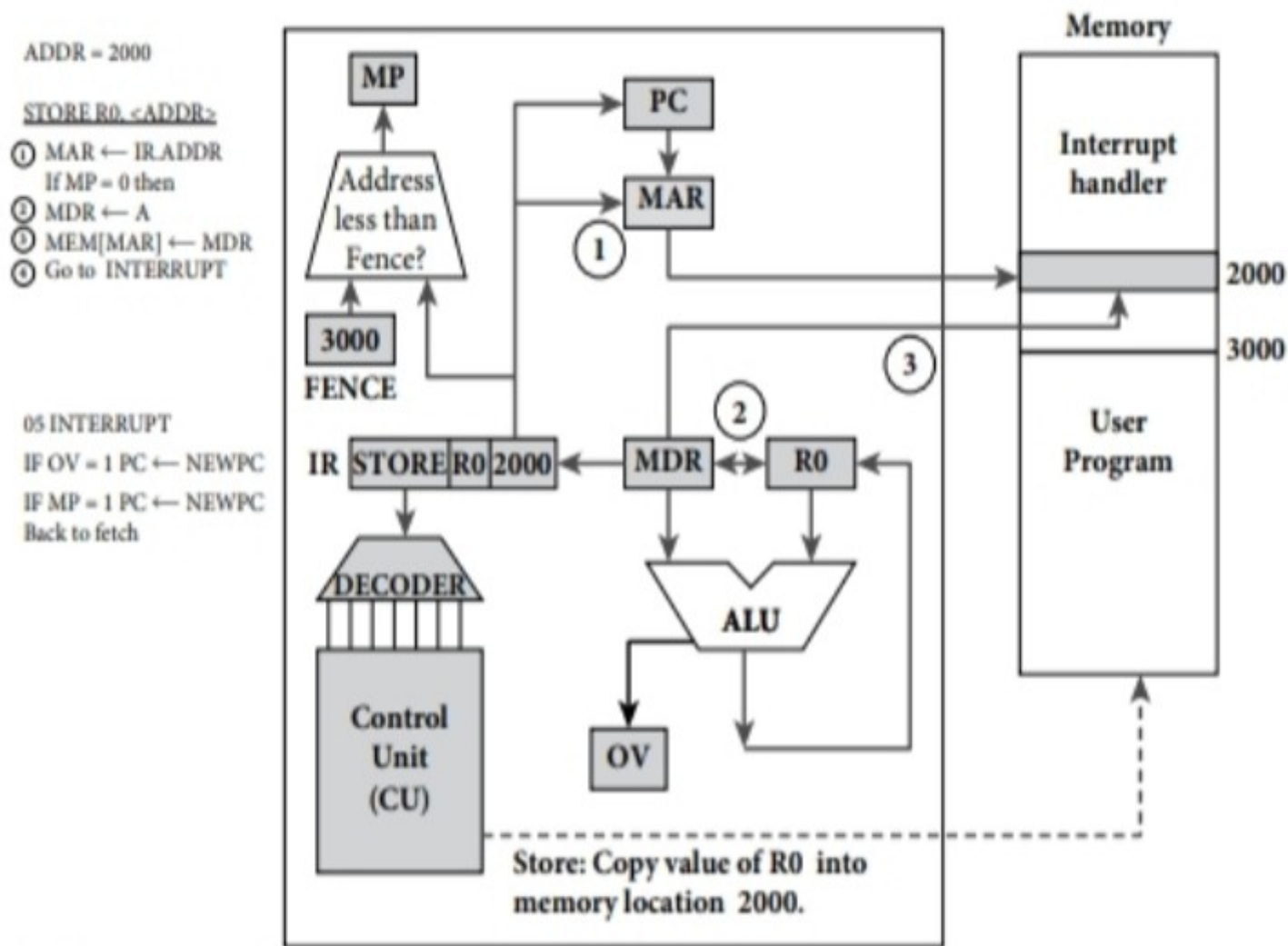


FIGURE 8.5 Memory protection mechanism.

VM/0, now with Memory Protection

If we want to load more than one program in memory, we need to give them bounds. Record the max extent of a process in a FENCE register.

Add some hardware to do a simple check of the requested address with the FENCE value.

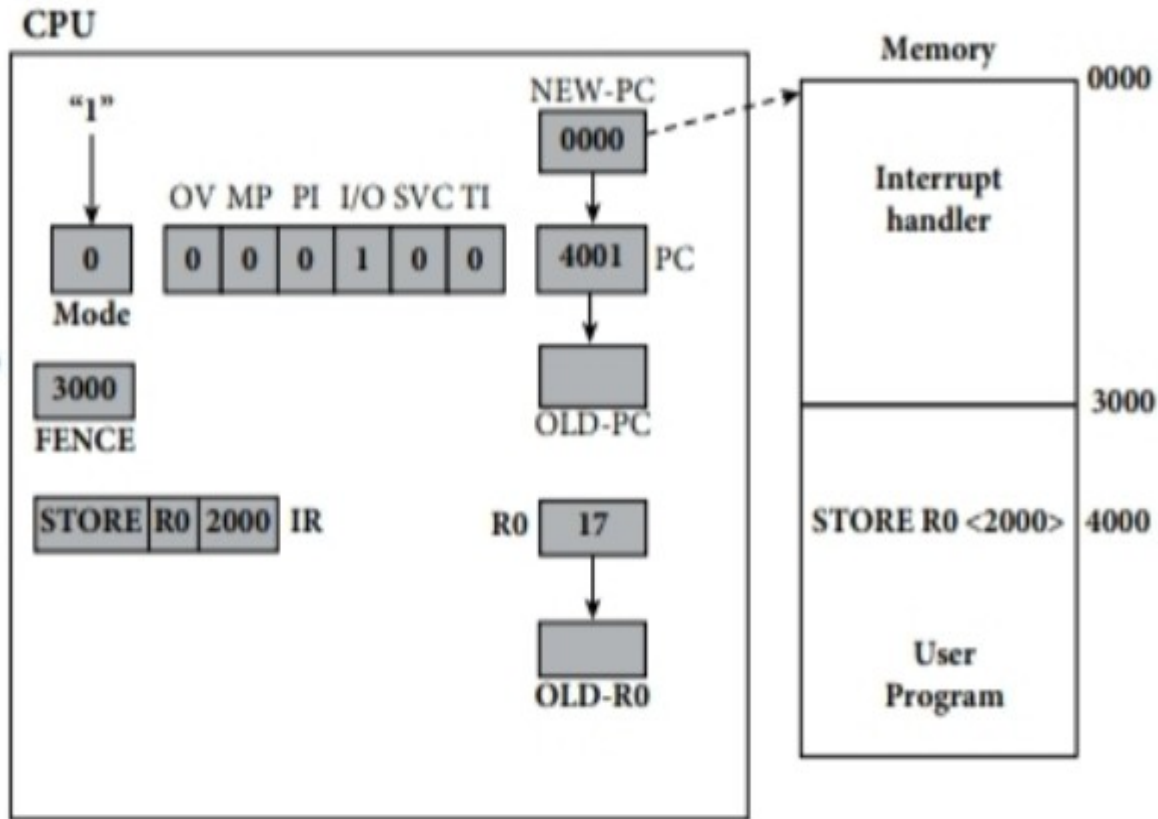
Add another flip-flop to flip when a request exceeds the bounds.

See a pattern? What should we do next?

A user program is running and CPU fetches an instruction located at address 4000

Remember that in the fetch step the PC is incremented by one.

Notice that the value in R0 is 17.



05 INTERRUPT

```
IF OV = 1 PC ← NEWPC; Mode ← 1
IF MP = 1 PC ← NEWPC; Mode ← 1
IF PI = 1 PC ← NEWPC; Mode ← 1
Back to fetch
```

Privileged Instructions have entered the chat!

Protecting the Fence Register

The Fence register can be overwritten by programs, allowing access to other processes' memory spaces. We need to prevent "ordinary process" code from manipulating it.

The Mode Bit and Privileged Execution Mode

Instructions are divided between Privileged and Unprivileged modes. Opcodes above a certain value are considered privileges. Set a Mode bit at runtime designating the mode the system is running in. If the Mode is unprivileged and the opcode trying to

A user program is running and while CPU is executing Fetch-Execute an I/O interrupt occurs.

Remember that in the fetch step the PC is incremented by one.

Notice that the value loaded in R0 is 17.

05 INTERRUPT

IF OV = 1 PC ← NEWPC ; Mode ← 1

IF MP = 1 PC ← NEWPC ; Mode ← 1

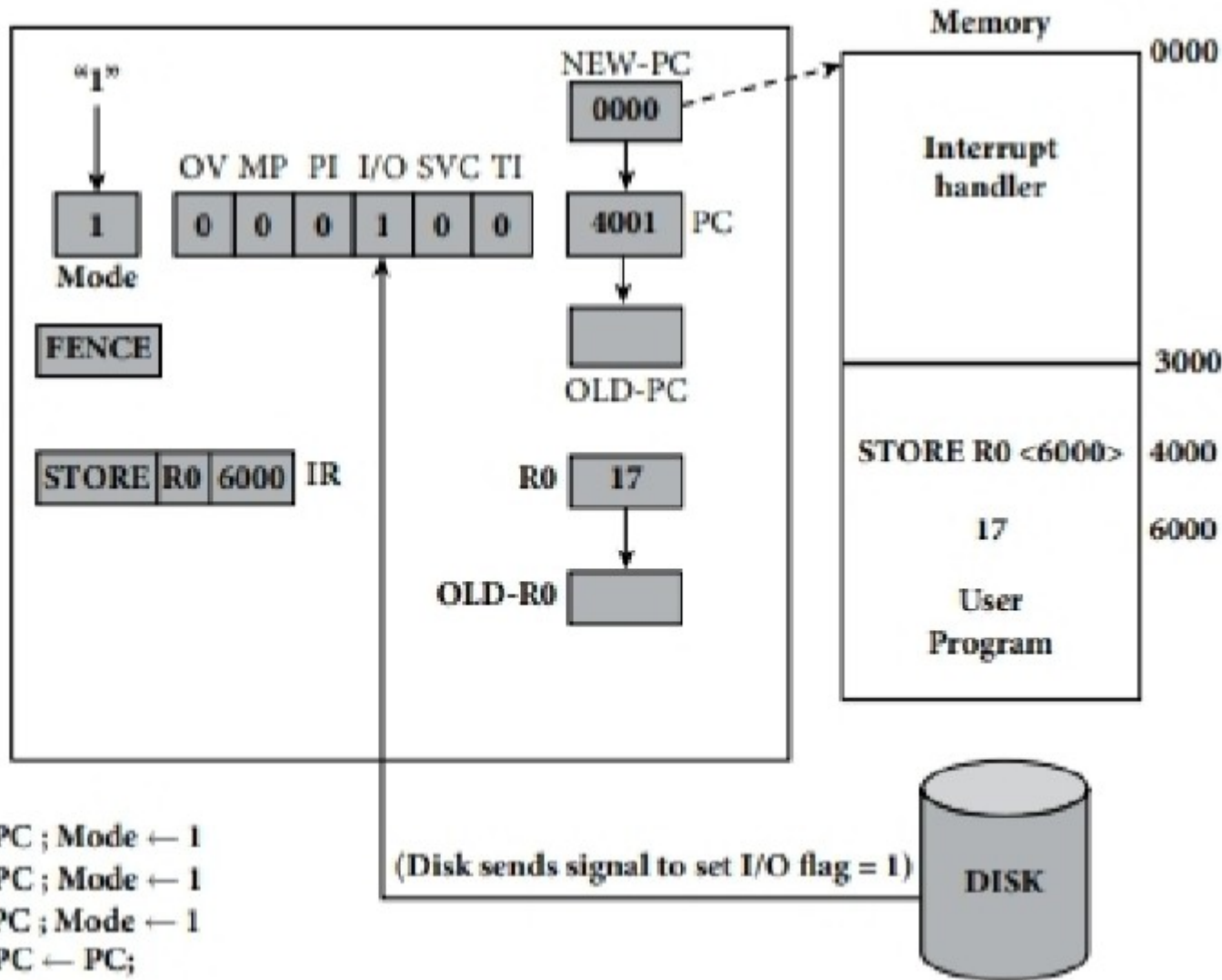
IF PI = 1 PC ← NEWPC ; Mode ← 1

IF I/O = 1 THEN OLDPC ← PC;

 OLD.R0 ← R0;

 PC ← NEWPC ; Mode ← 1;

Back to fetch



VM/0 with I/O Interrupt

Just like before, we add another flip-flip, another register to store PC information and a register to store the previous data stored in R0.

Now we can switch between the I/O handler routine and the calling program. The new registers allow us to save program state, run the I/O, and then restore.

THE PATTERN CONTINUES

- By now you've seen the pattern. We add more flip-flops, registers, and handler routines in system memory.
- Instead of an array of flip-flops, we have one unified register, the Interrupt Handler Vector.

SYSTEM CALLS

We can now allow programs to make requests of the system and change between execution modes. And now the space that was previously populated by a handful of routines has expanded greatly into a library of functions that are available to processes.

TIMERS

Timer interrupts now allow the system to take control and, as we'll see in a future episode, interrupt a running process so that another can be run in its place.

Interrupt Handler Vector						
IHV	Address Overflow Handler	Address MP Handler	Address PI Handler	Address Timer Handler	Address I/O Handler	Address SVC Handler
	0	1	2	3	4	5